

Rodge Rebeca

October 21st, 2025

Description

In this project, the wildcard server will need to be running initially and will act as a TCP server that can accept up to 5 TCP connection requests at a time. The server's socket will also be defined and binded to allow clients to connect to the server socket and utilize the wildcard matching process.

On the client side, the server port and IP address (localhost) is defined and a client socket will be created to connect to the server socket. Once a connection is established, the user is prompted to place their input for a query where input is also lowercased and trimmed for trailing white space. Client will print codes based on HTTP codes if: server is not able to be connected, successful query, and successful connection but no matches or content sent back.

The application protocol will work as follows such that a successful TCP connection is established between the server and client. A new thread is created where the sent client byte data is decoded and will iterate through every entry in the english dictionary txt file. The algorithm essentially will compare the query and current iterated entry's each character to see if they match character-wise or "?" exists. A no match will occur if there either exists a mismatched character or the lengths do not match. Once all matches are found, these are placed on an accumulated string to be encoded and sent back to the client.

Tradeoffs

One of the design tradeoffs was determining how the English dictionary would be either hard-coded or should be read from a txt file. Hard-coding english dictionary entries would simplify the server startup process and utilize less resources since it would not need to do an extra I/O handling and would reduce additional dependency. But in the real world, vocabulary is a constantly evolving and changing concept so in the scenario that new words are introduced, it would take more time checking for missing new words and it would be more scalable to be able to provide an external file. The only issue with this is that there would be some processing overhead to handle I/O handling but it is worth it since it would be best industry practice to do so.

Another tradeoff was using TCP vs UDP. Although the task was to use TCP socket programming, we could also rewrite the code to be a UDP server and client. The advantages of UDP would allow the server to serve more clients with less processing overhead since it would not need to maintain and store connections with clients. The only issue with this is that data can be prone to corruption and loss. Given the nature and small scale of the project, TCP would be best to ensure that data is being reliably delivered.

Extensions

The provided English dictionary txt file contains 69,903 entries. Where each entry will need to be checked and each character in each entry will need to be checked. This would result in a runtime of $O(n * m)$ where n represents the amount of words in the dictionary and m represents the longest word in length. We could reduce the amount of checking by either having a dictionary key-value pair where the key is the length of a set of words and the value would be a list of all words with length of key value reducing the amount of comparisons significantly.

Test Cases

In order to verify functionality, we will validate the following test cases:

1. Query without “?”
2. Query with 1 “?”
3. Query with all “?”
4. Client quit
5. Query with no matches
6. Offline server
7. Query with excess spaces and capitalization
8. 2 Simultaneous client connections

Query without “?”

```
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard % python threaded_client.py
send message (or 'quit'): apple
HTTP 200 OK
From Wildcard Server, matches for 'apple' include:
apple
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard %
```

```
rodgerebeca@macbookpro threaded_wildcard % python threaded_server.py
Wildcard Server is Listening...
Connection established with 127.0.0.1:58302
Connection closed with 127.0.0.1:58302
```

Query with 1 “?”

```
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard % python threaded_client.py
send message (or 'quit'): app?e
HTTP 200 OK
From Wildcard Server, matches for 'app?e' include:
apple
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard %
```

```
rodgerebeca@macbookpro threaded_wildcard % python threaded_server.py
Wildcard Server is Listening...
Connection established with 127.0.0.1:58303
Connection closed with 127.0.0.1:58303
```

Query with all “?”

```
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard % ls
thread-server.py      threaded_server.py
threaded_client.py    wordlist.txt
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard % ls
thread-server.py      threaded_server.py
threaded_client.py    wordlist.txt
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard % python threaded_client.py
send message (or 'quit'): ???
HTTP 200 OK
From Wildcard Server, matches for '???' include:
aba
abc
aby
ace
act
acu
add
ade
adj
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard %
```

```
rodgerebeca@macbookpro threaded_wildcard % python threaded_server.py
Wildcard Server is Listening...
Connection established with 127.0.0.1:58305
Connection closed with 127.0.0.1:58305
```

Client Quit

```
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard % python threaded_client.py
send message (or 'quit'): quit
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard %
```

```
rodgerebeca@macbookpro threaded_wildcard % python threaded_server.py
Wildcard Server is Listening...
Connection established with 127.0.0.1:58305
Connection closed with 127.0.0.1:58305
Connection established with 127.0.0.1:58310
Connection closed with 127.0.0.1:58310
```

Query with no matches

```
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard % python threaded_client.py
send message (or 'quit'): 12345
HTTP 204 No Content
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard %
```

```
rodgerebeca@macbookpro threaded_wildcard % python threaded_server.py
Wildcard Server is Listening...
Connection established with 127.0.0.1:58315
Connection closed with 127.0.0.1:58315
```

Offline Server

```
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard % python threaded_client.py
Traceback (most recent call last):
  File "/Users/rodgerebeca/projects/CSE3300/project1/threaded_wildcard/threaded_client.py", line 22, in tcp_client
    client_socket.connect((server_name, server_port))
    ~~~~~^~~~~~
ConnectionRefusedError: [Errno 61] Connection refused

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "/Users/rodgerebeca/projects/CSE3300/project1/threaded_wildcard/threaded_client.py", line 47, in <module>
    tcp_client()
    ~~~~~^~~~~~
  File "/Users/rodgerebeca/projects/CSE3300/project1/threaded_wildcard/threaded_client.py", line 24, in tcp_client
    raise ConnectionError('HTTP 404 not Found')
ConnectionError: HTTP 404 not Found
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard %
```

Query with excess space and capitalization

```
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard % clear
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard % python threaded_client.py
send message (or 'quit'): hELLo
HTTP 200 OK
From Wildcard Server, matches for 'hello' include:
hello
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard %
```

```
rodgerebeca@macbookpro threaded_wildcard % python threaded_server.py
Wildcard Server is Listening...
Connection established with 127.0.0.1:58326
Connection closed with 127.0.0.1:58326
```

2 Simultaneous Client Connections

```
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard % python threaded_client.py
send message (or 'quit'): worl?
HTTP 200 OK
From Wildcard Server, matches for 'worl?' include:
world
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard %
```

```
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard % python threaded_client.py
send message (or 'quit'): hell?
HTTP 200 OK
From Wildcard Server, matches for 'hell?' include:
hello
hells
rodgerebeca@Rodges-MacBook-Pro threaded_wildcard %
```

```
rodgerebeca@macbookpro threaded_wildcard % python threaded_server.py
Wildcard Server is Listening...
Connection established with 127.0.0.1:58327
Connection established with 127.0.0.1:58328
Connection closed with 127.0.0.1:58327
Connection closed with 127.0.0.1:58328
```